

2026 on Neural Prefetching for CPU Caches

AI Base Prefetcher for L2 Cache —— *with LSTM*

Stride

Streamer

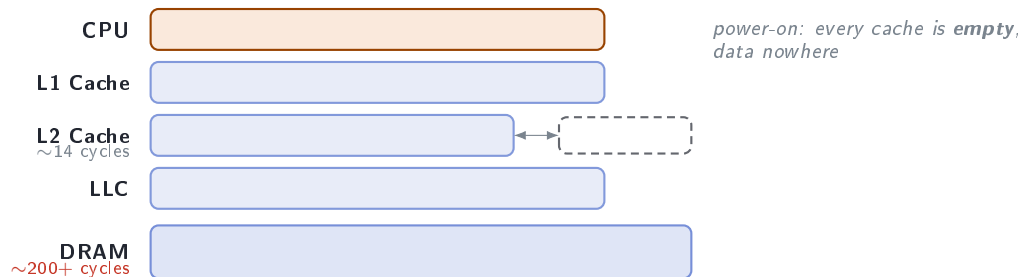
AMPM

SPP

four hand-written prefetchers · one LSTM recipe · judged only in full-system replay

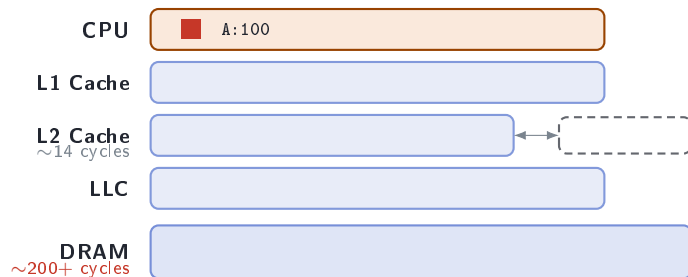
Start With an Empty Cache

one load's journey through the machine



Start With an Empty Cache

one load's journey through the machine

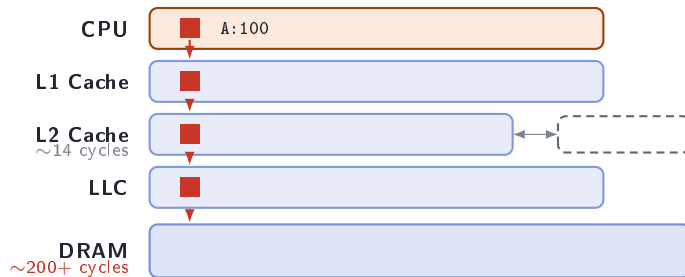


*power-on: every cache is **empty**,
data nowhere*

the core asks for line 100

Start With an Empty Cache

one load's journey through the machine



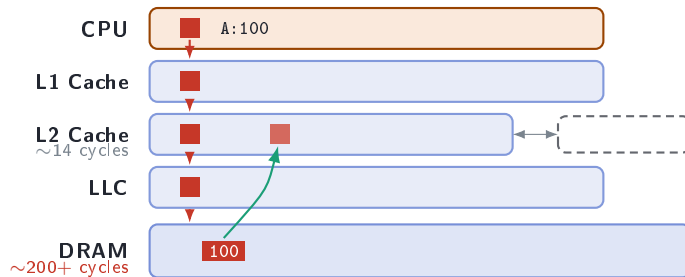
*power-on: every cache is **empty**,
data nowhere*

the core asks for line 100

not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

Start With an Empty Cache

one load's journey through the machine



power-on: every cache is **empty**, data nowhere

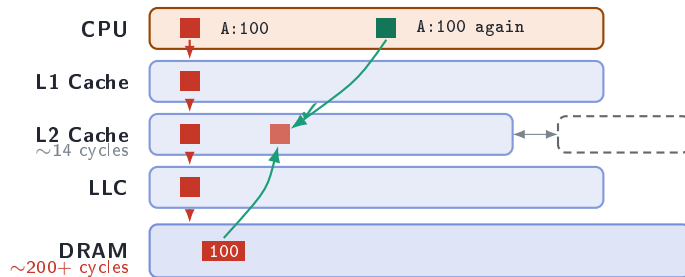
the core asks for line 100

not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

found only in DRAM;
L2 fills ~**200+ cycles** later:
correct, but the core waited

Start With an Empty Cache

one load's journey through the machine



power-on: every cache is **empty**,
data nowhere

the core asks for line 100

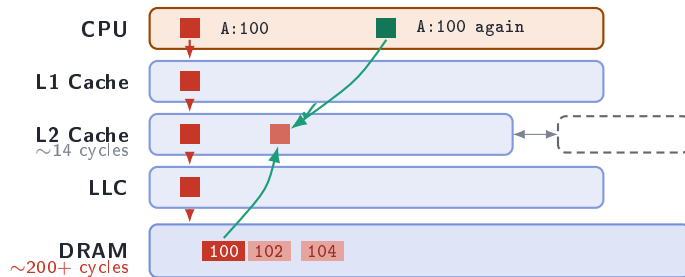
not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

found only in DRAM;
L2 fills ~**200+ cycles** later:
correct, but the core waited

✓ **Temporal Locality**
what you touched comes back: a hit

Start With an Empty Cache

one load's journey through the machine



power-on: every cache is **empty**,
data nowhere

the core asks for line 100

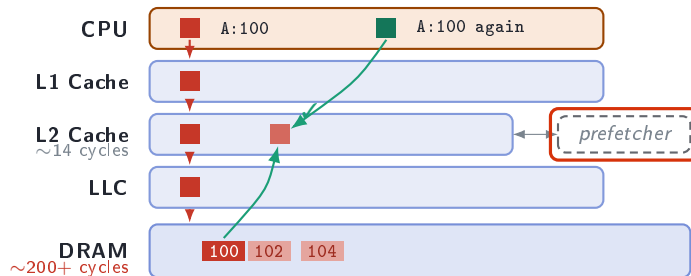
not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

found only in DRAM;
L2 fills ~**200+ cycles** later:
correct, but the core waited

- ✓ **Temporal Locality**
what you touched comes back: a hit
- ✓ **Spatial Locality**
its neighbors are asked for *next*

Start With an Empty Cache

one load's journey through the machine



The future is written in advance — someone should fetch those neighbors before the core asks.

*power-on: every cache is **empty**, data nowhere*

the core asks for line 100

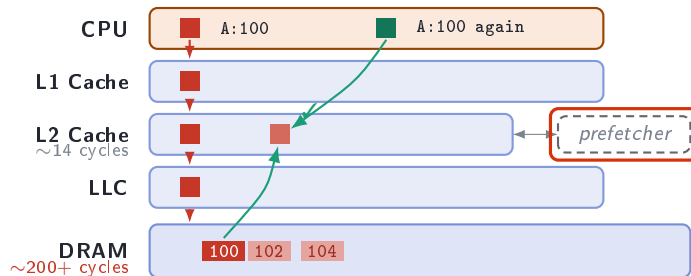
not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

found only in DRAM;
L2 fills **~200+ cycles** later:
correct, but the core waited

- ✓ **Temporal Locality**
what you touched comes back: a hit
- ✓ **Spatial Locality**
its neighbors are asked for *next*

Start With an Empty Cache

one load's journey through the machine



The future is written in advance — someone should fetch those neighbors before the core asks.

power-on: every cache is **empty**, data nowhere

the core asks for line 100

not in L1, L2, or LLC:
the first touch hits *nowhere*:
a compulsory miss

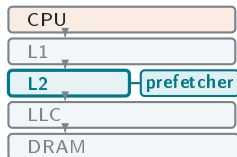
found only in DRAM;
L2 fills **~200+ cycles** later:
correct, but the core waited

- ✓ **Temporal Locality**
what you touched comes back: a hit
- ✓ **Spatial Locality**
its neighbors are asked for *next*

► *Fill the slot. First, with a machine you can write by hand.*

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



INSIDE THE PREFETCHER · five visits by one load

A:100 B:40

MISS

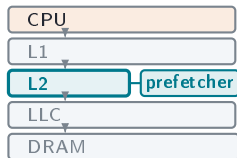
ITS WHOLE STATE · one lane per PC

PC	last line	last stride
0x401c	102	+2
0x77f0	44	+4

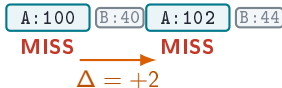
64 lanes · LRU · exact-PC lookup · two numbers

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



INSIDE THE PREFETCHER · five visits by one load



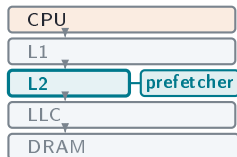
ITS WHOLE STATE · one lane per PC

PC	last line	last stride
0x401c	102	+2
0x77f0	44	+4

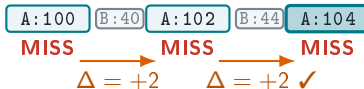
64 lanes · LRU · exact-PC lookup · two numbers

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



INSIDE THE PREFETCHER · five visits by one load



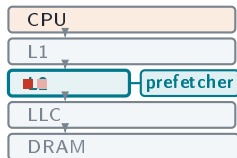
ITS WHOLE STATE · one lane per PC

PC	last line	last stride
▶ 0x401c	102	+2
0x77f0	44	+4

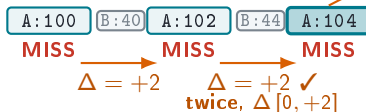
64 lanes · LRU · exact-PC lookup · two numbers

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



INSIDE THE PREFETCHER · five visits by one load



ITS WHOLE STATE · one lane per PC

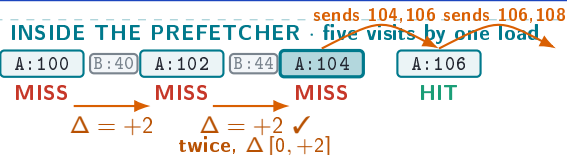
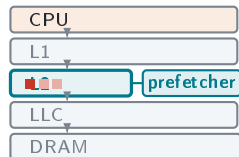
PC	last line	last stride
▶ 0x401c	102	+2
0x77f0	44	+4

on A:104: (102, +2) → (104, +2)

64 lanes · LRU · exact-PC lookup · two numbers

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



ITS WHOLE STATE · one lane per PC

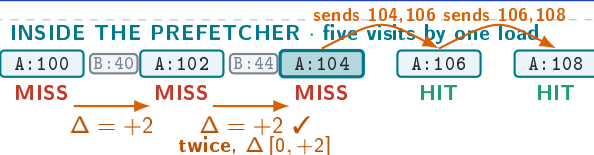
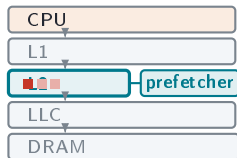
PC	last line	last stride
▶ 0x401c	102	+2
0x77f0	44	+4

on A:104: (102, +2) → (104, +2)

64 lanes · LRU · exact-PC lookup · two numbers

A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



ITS WHOLE STATE · one lane per PC

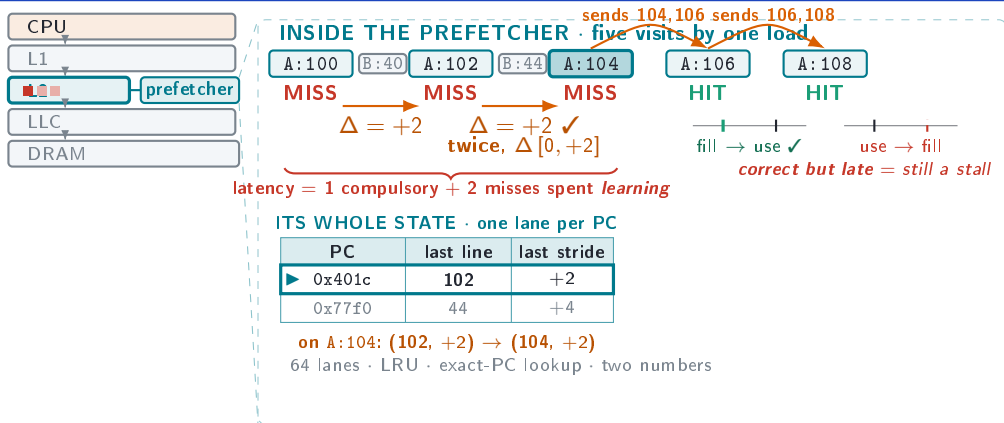
PC	last line	last stride
▶ 0x401c	102	+2
0x77f0	44	+4

on A:104: (102, +2) → (104, +2)

64 lanes · LRU · exact-PC lookup · two numbers

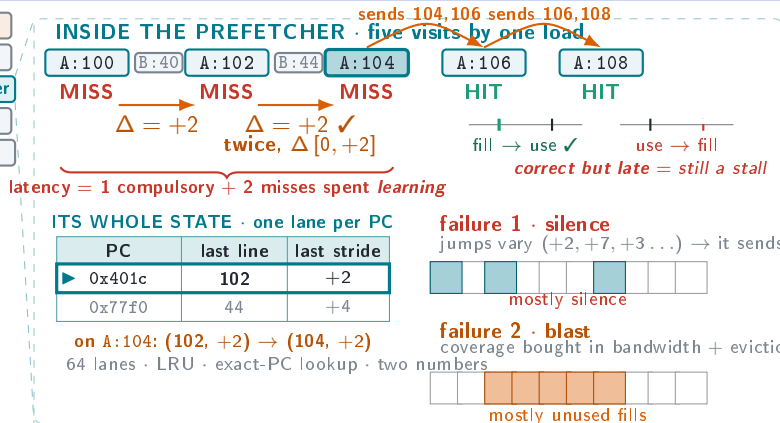
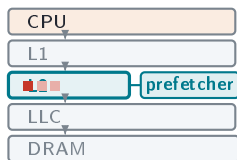
A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



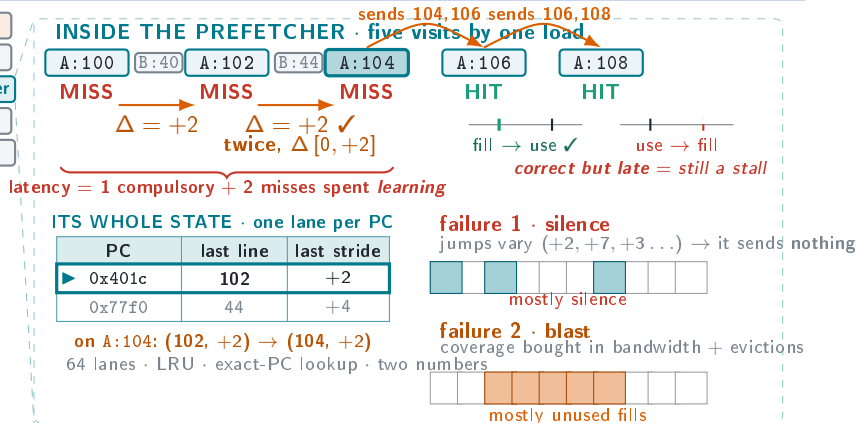
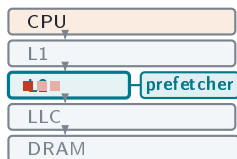
A Prefetcher by Hand: Stride

two numbers, one comparison — and its bill



A Prefetcher by Hand: Stride

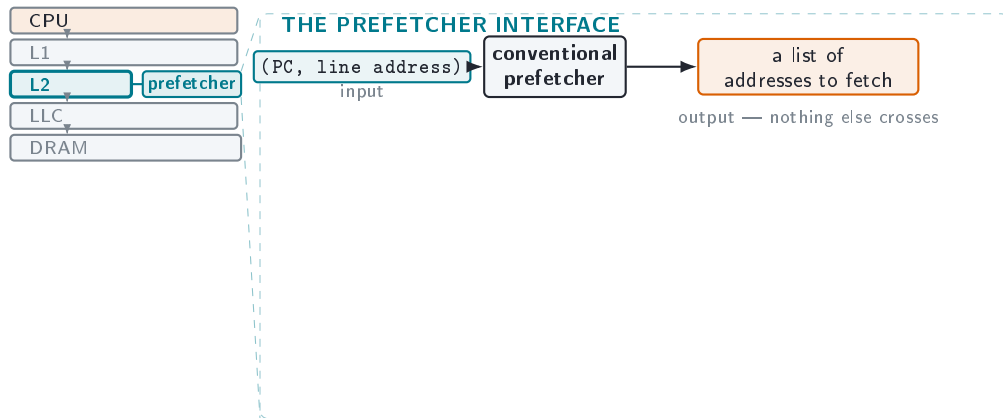
two numbers, one comparison — and its bill



► judged on three things: • what it **sends** • what that **removes** • does it land **in time**?

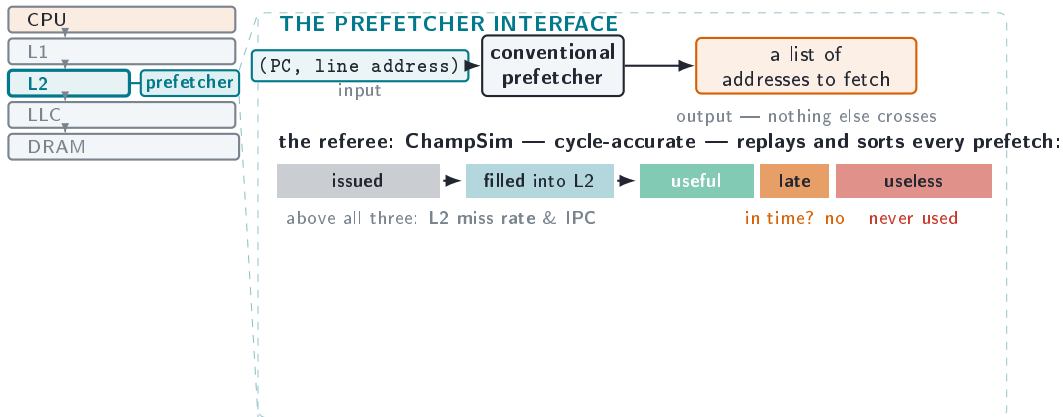
The Prefetcher Interface, and the Referee

any box is judged the same way



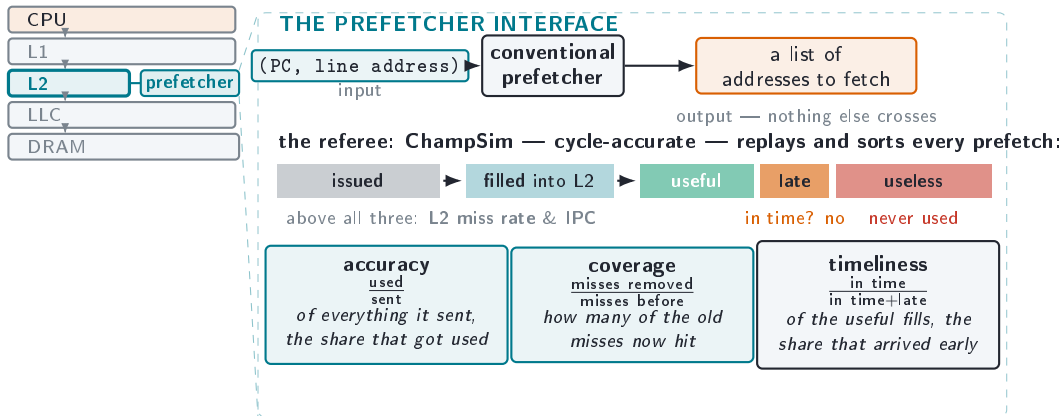
The Prefetcher Interface, and the Referee

any box is judged the same way



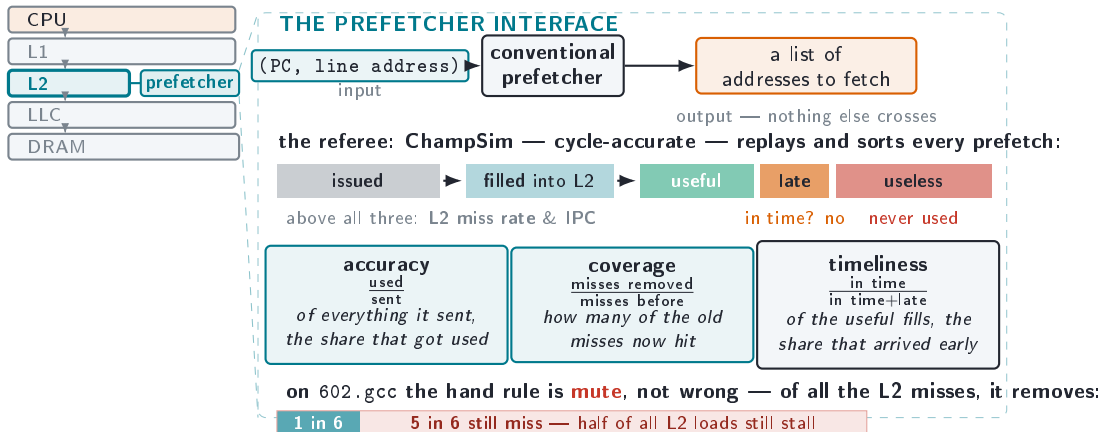
The Prefetcher Interface, and the Referee

any box is judged the same way



The Prefetcher Interface, and the Referee

any box is judged the same way

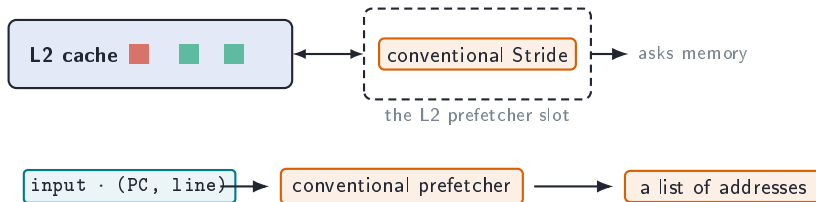


► A human froze the **window** (the last two events) and the **trigger** (seen twice) — next, let a model learn both.

A New Occupant: an ML Prefetcher

same interface, same input, same referee

the same slot the machine has had all along — only the occupant changes:



A New Occupant: an ML Prefetcher

same interface, same input, same referee

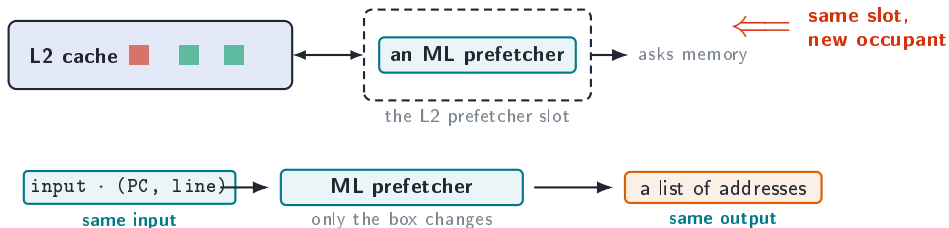
the same slot the machine has had all along — only the occupant changes:



A New Occupant: an ML Prefetcher

same interface, same input, same referee

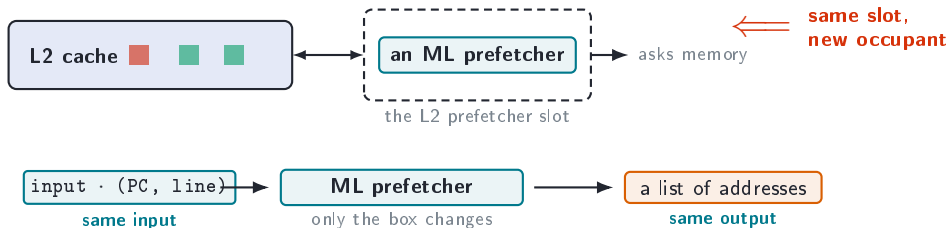
the same slot the machine has had all along — only the occupant changes:



A New Occupant: an ML Prefetcher

same interface, same input, same referee

the same slot the machine has had all along — only the occupant changes:



- **more versatile** — one recipe re-targets any hand-written rule
- **more flexible** — history window and trigger are *learned*, not frozen
- **better prediction** — judged where it counts: full-system replay

A New Occupant: an ML Prefetcher

same interface, same input, same referee

the same slot the machine has had all along — only the occupant changes:



- **more versatile** — one recipe re-targets any hand-written rule
- **more flexible** — history window and trigger are *learned*, not frozen
- **better prediction** — judged where it counts: full-system replay

all it ever sees is (PC, address), one event at a time — what can it learn from so little?

A Network That Remembers

the recurrent neural network (RNN)

history A 100 102 104 next = 106

history B 96 100 104 next = 108

A Network That Remembers

the recurrent neural network (RNN)

history A 100 102 104 next = 106

history B 96 100 104 next = 108

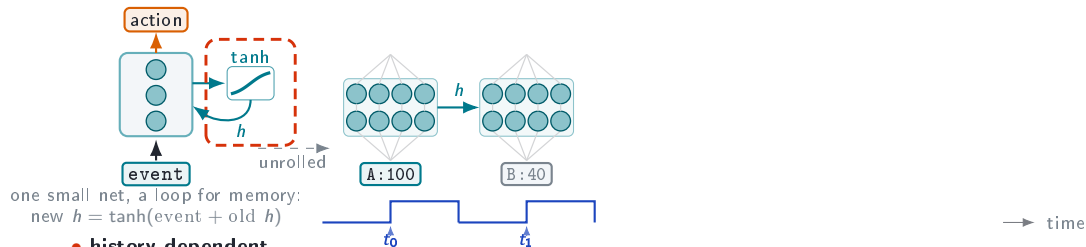
- **history dependent**
- a **time-sequence** predictor
- **training** decides the h -window

A Network That Remembers

the recurrent neural network (RNN)

history A [100] [102] [104] next = 106

history B [96] [100] [104] next = 108



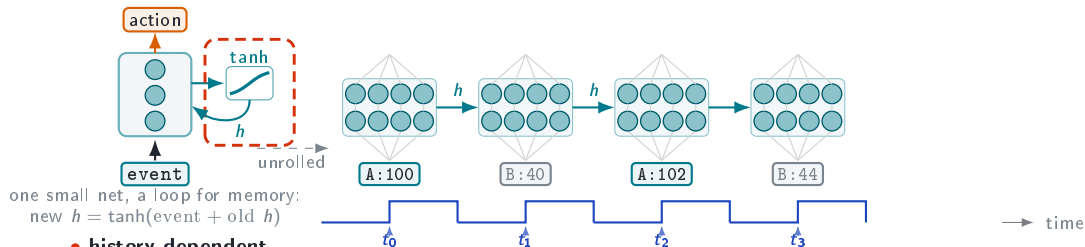
- **history dependent**
- **a time-sequence predictor**
- **training** decides the h -window

A Network That Remembers

the recurrent neural network (RNN)

history A 100 102 104 next = 106

history B 96 100 104 next = 108



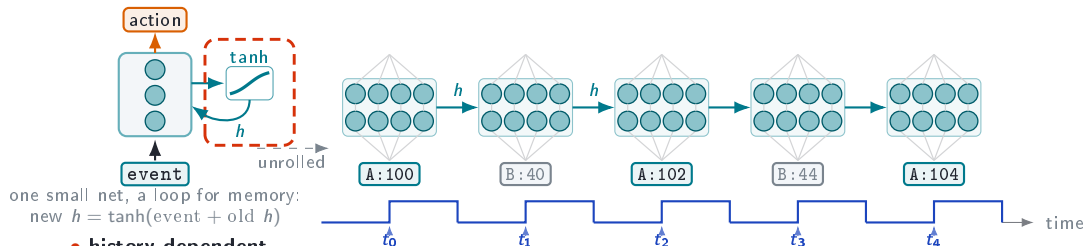
- history dependent
- a time-sequence predictor
- training decides the h -window

A Network That Remembers

the recurrent neural network (RNN)

history A [100] [102] [104] next = 106

history B [96] [100] [104] next = 108



- history dependent
- a time-sequence predictor
- training decides the h -window

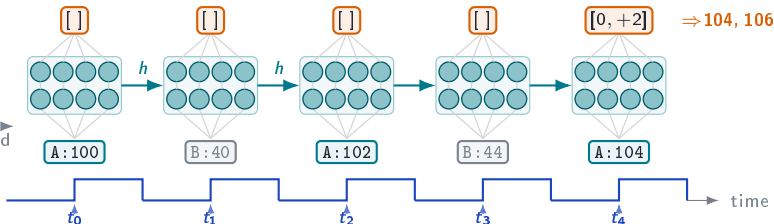
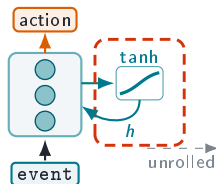
A Network That Remembers

the recurrent neural network (RNN)

history A 100 102 104 next = 106

history B 96 100 104 next = 108

label = what the rule did:



one small net, a loop for memory:
new $h = \tanh(\text{event} + \text{old } h)$

- **history dependent**
- **a time-sequence predictor**
- **training** decides the h -window

the same weights at every step — what changes is the carried h

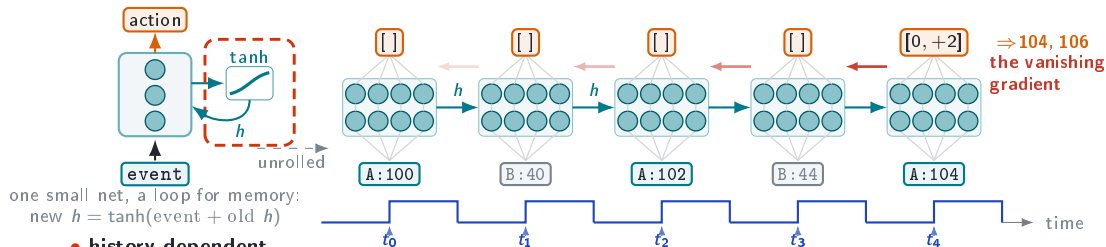
A Network That Remembers

the recurrent neural network (RNN)

history A 100 102 104 next = 106

history B 96 100 104 next = 108

label = what the rule did:



one small net, a loop for memory:
new $h = \tanh(\text{event} + \text{old } h)$

- **history dependent**
- **a time-sequence predictor**
- **training** decides the h -window

the same weights at every step — what changes is the carried h

training credit flows backwards through every overwrite of h — each step multiplies it **smaller**, so distant events teach almost **nothing**: the history window is left to **chance**

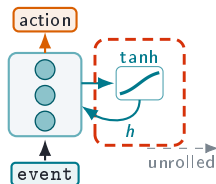
A Network That Remembers

the recurrent neural network (RNN)

history A [100] [102] [104] next = 106

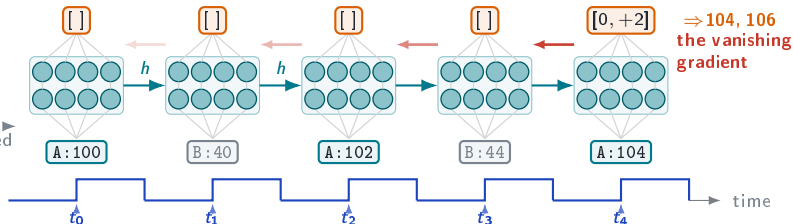
history B [96] [100] [104] next = 108

label = what the rule did:



one small net, a loop for memory:
new $h = \tanh(\text{event} + \text{old } h)$

- history dependent
- a time-sequence predictor
- training decides the h -window



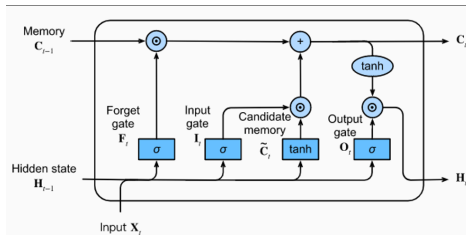
the same weights at every step — what changes is the carried h

training credit flows backwards through every overwrite of h — each step multiplies it **smaller**, so distant events teach almost **nothing**: the history window is left to **chance**

► The fix for the fade: **gates** — let the network learn what to keep, and for how long.

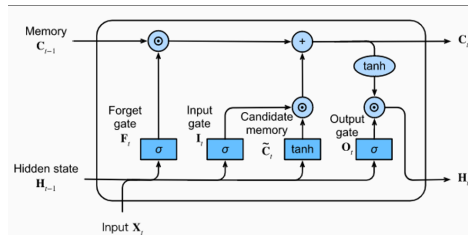
Long Short-Term Memory (LSTM) *learn how long to remember: keep, write, reveal*

Same current event, different history $\Rightarrow$ different answer. The answer lives in the history.



Long Short-Term Memory (LSTM) *learn how long to remember: keep, write, reveal*

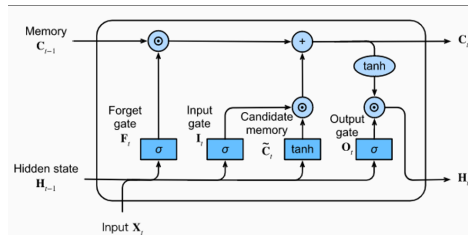
Same current event, different history $\Rightarrow$ different answer. The answer lives in the history.



- each gate = a tiny **learned** layer over [previous memory, this event]

Long Short-Term Memory (LSTM) *learn how long to remember: keep, write, reveal*

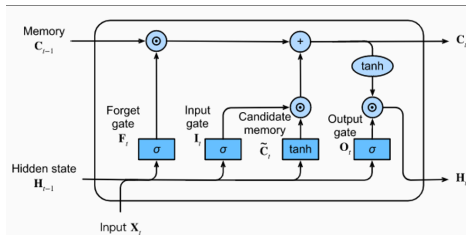
Same current event, different history $\Rightarrow$ different answer. The answer lives in the history.



- each gate = a tiny **learned** layer over [previous memory, this event]
- nobody defines “stride” — the gates learn **how long** each fact is **worth holding**

Long Short-Term Memory (LSTM) *learn how long to remember: keep, write, reveal*

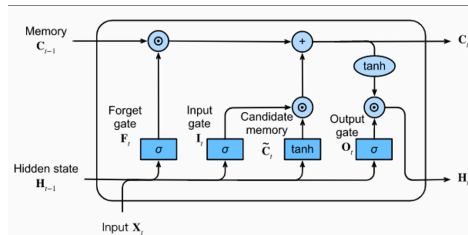
Same current event, different history $\Rightarrow$ different answer. The answer lives in the history.



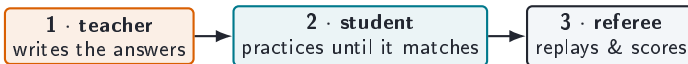
- each gate = a tiny **learned** layer over [previous memory, this event]
- nobody defines “stride” — the gates learn **how long each fact is worth holding**
- $h = H$ numbers that **together encode what the network has learned to remember about the history**
— h8 keeps **8** numbers, h128 keeps **128**; the *only* knob we sweep

Long Short-Term Memory (LSTM) *learn how long to remember: keep, write, reveal*

Same current event, different history $\Rightarrow$ different answer. The answer lives in the history.



- each gate = a tiny **learned** layer over [previous memory, this event]
- nobody defines “stride” — the gates learn **how long each fact is worth holding**
- $h = H$ numbers that **together encode what the network has learned to remember about the history**
— h8 keeps **8** numbers, h128 keeps **128**; the *only* knob we sweep



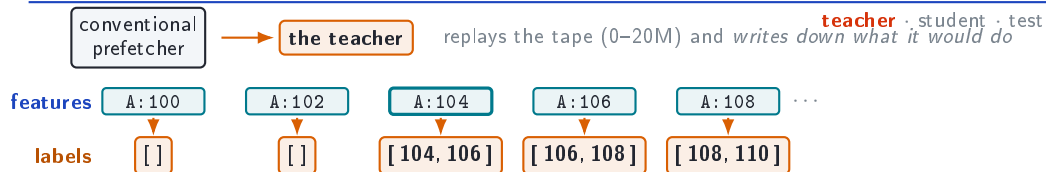
Creating Labels · the Teacher

a label's only job: score the guess



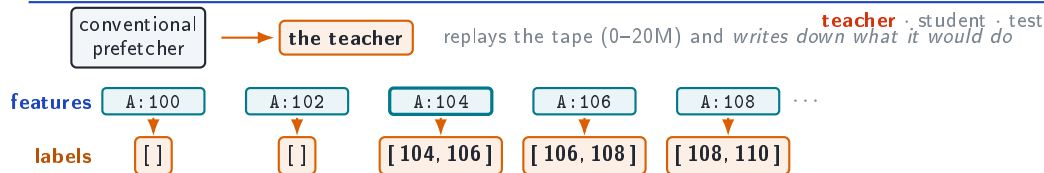
Creating Labels · the Teacher

a label's only job: score the guess

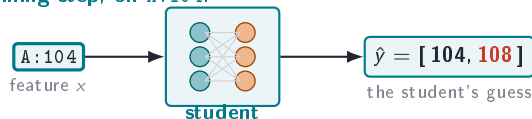


Creating Labels · the Teacher

a label's only job: score the guess

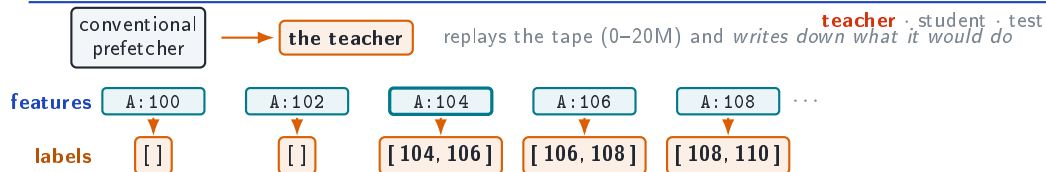


one training step, on A:104:

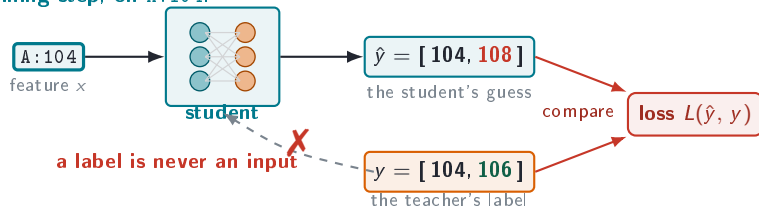


Creating Labels · the Teacher

a label's only job: score the guess

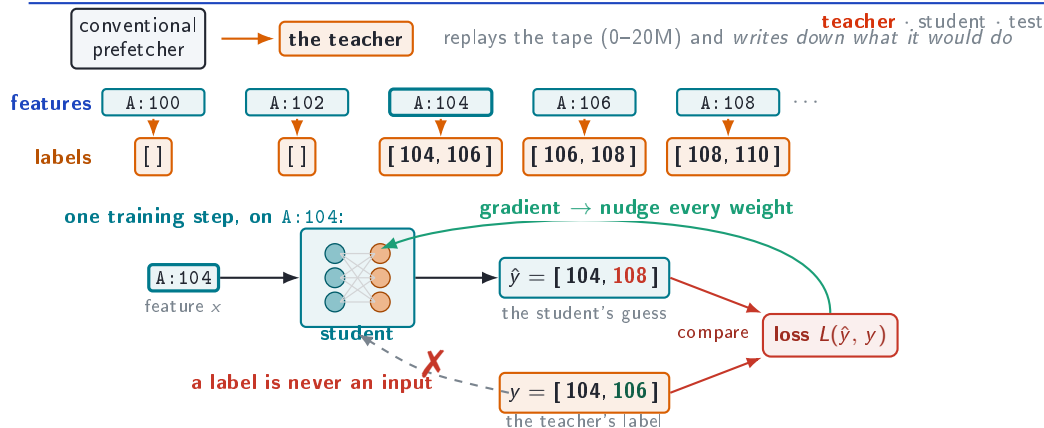


one training step, on A:104:



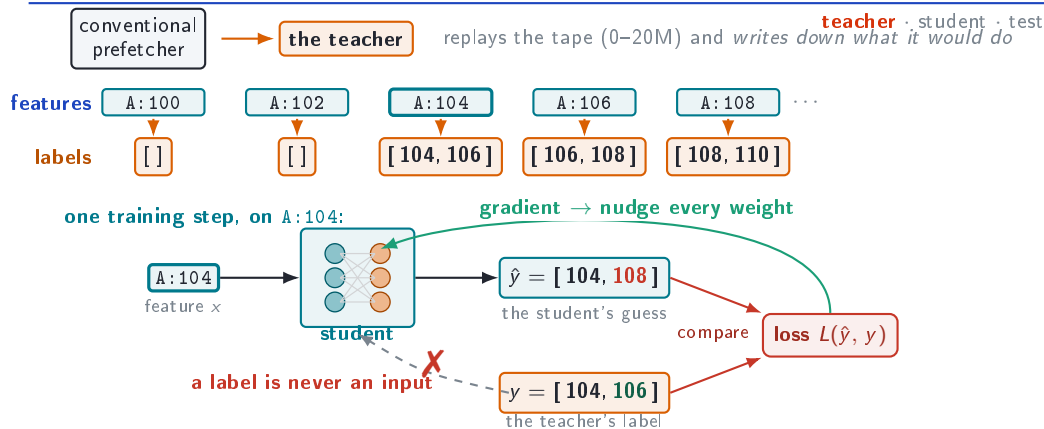
Creating Labels · the Teacher

a label's only job: score the guess



Creating Labels · the Teacher

a label's only job: score the guess



► The label enters only the loss — and the loss teaches two quieter lessons: when to stay silent, and how many to send.

Silent or Act · a Learned Decision the teacher waits; the student learns when to move

teacher · student · test

why is the teacher *silent* 84% of the time? the rule waits to see a stride twice:



most events look like these $\Rightarrow$ silent

Silent or Act · a Learned Decision the teacher waits; the student learns when to move

teacher · student · test

why is the teacher *silent* 84% of the time? the rule waits to see a stride twice:



teacher, on 404,219 events:



Silent or Act · a Learned Decision *the teacher waits; the student learns when to move*

why is the teacher *silent* 84% of the time? the rule waits to see a stride twice: teacher · student · test



teacher, on 404,219 events:



student h16, same events:



Silent or Act · a Learned Decision *the teacher waits; the student learns when to move*

teacher · student · test

why is the teacher *silent* 84% of the time? the rule waits to see a stride twice:



teacher, on 404,219 events:



student h16, same events:



the student **breaks** the teacher's caution — it acts on **40.5%** of silent events (vs 1.6% back)
 and acting where the teacher was mute is where **coverage grows: 17% → 58%**

Silent or Act · a Learned Decision *the teacher waits; the student learns when to move*

why is the teacher *silent* 84% of the time? the rule waits to see a stride twice: teacher · student · test



most events look like these $\Rightarrow$ silent

teacher, on 404,219 events:



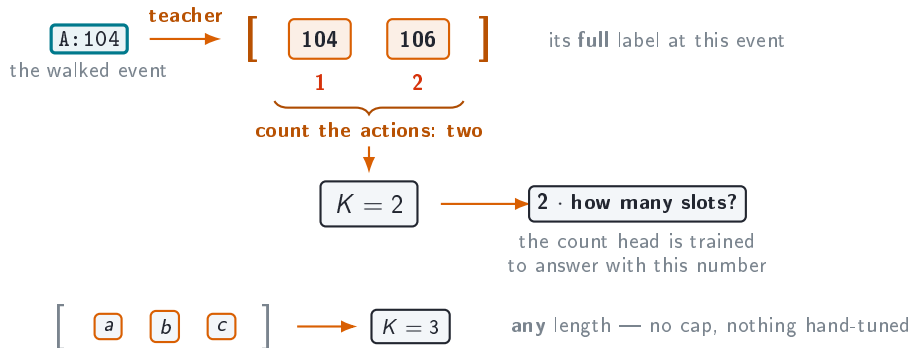
student h16, same events:



the student **breaks** the teacher's caution — it acts on **40.5%** of silent events (vs 1.6% back)
and acting where the teacher was mute is where **coverage grows: 17% $\rightarrow$ 58%**

► Lesson two hides in the label's length: how many to send.

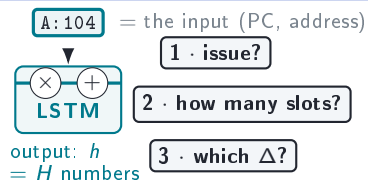
Why $K=2$? · the Count Is a Label Too *no cap, no curve — read it off the teacher*



► *Restraint and count, both read off the teacher. Now close the loop: answer, grade, nudge.*

Close-Loop Training · the Student

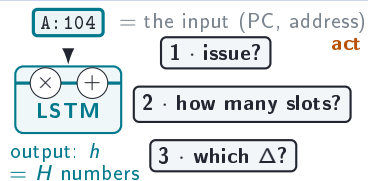
answer, grade, nudge every weight — repeat



teacher · **student** · test

Close-Loop Training · the Student

answer, grade, nudge every weight — repeat

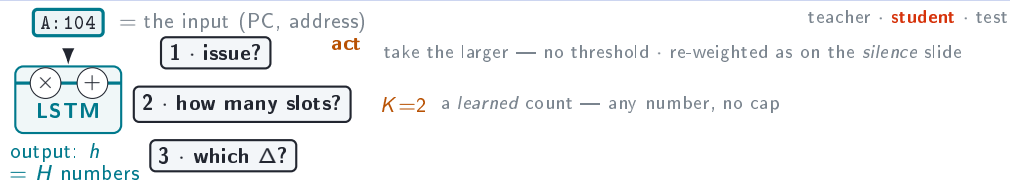


teacher · **student** · test

take the larger — no threshold · re-weighted as on the *silence* slide

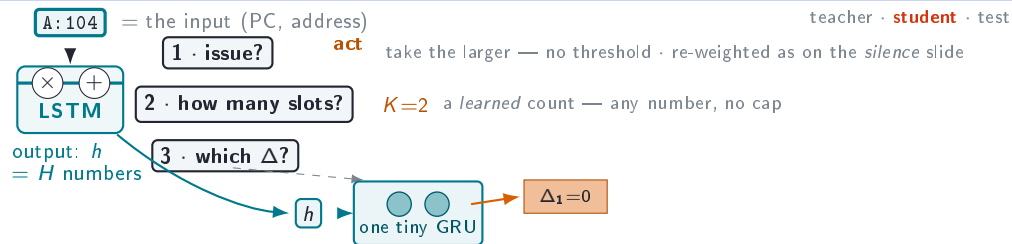
Close-Loop Training · the Student

answer, grade, nudge every weight — repeat



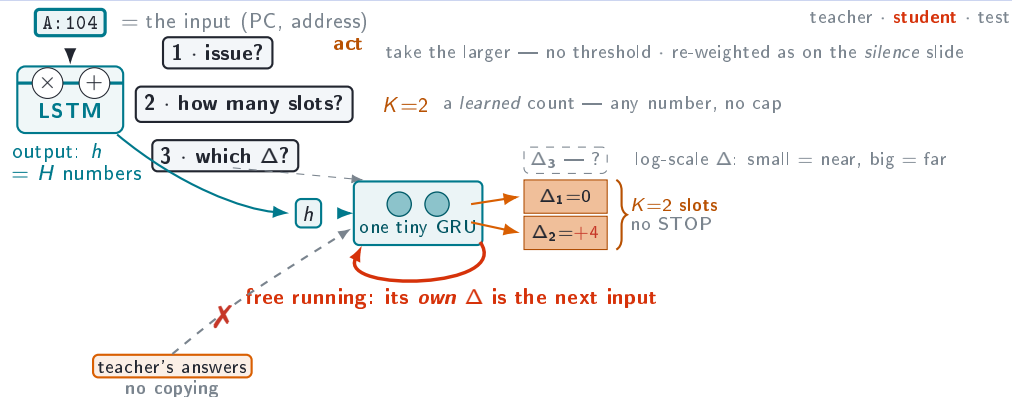
Close-Loop Training · the Student

answer, grade, nudge every weight — repeat



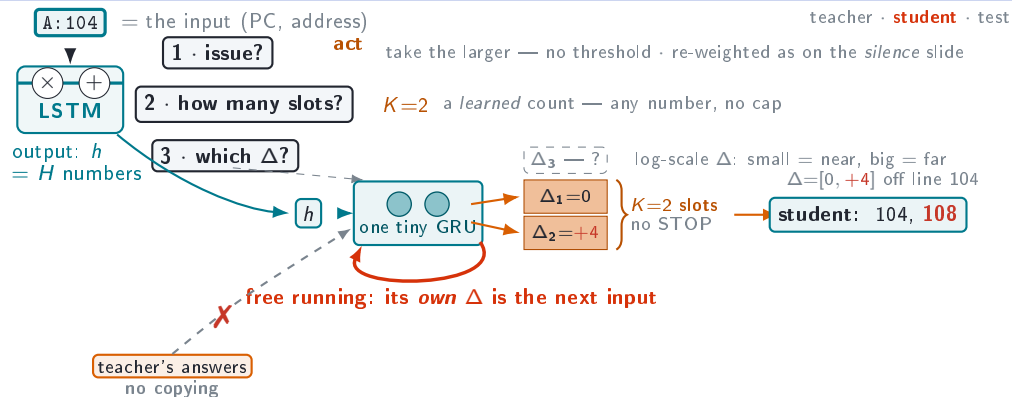
Close-Loop Training · the Student

answer, grade, nudge every weight — repeat



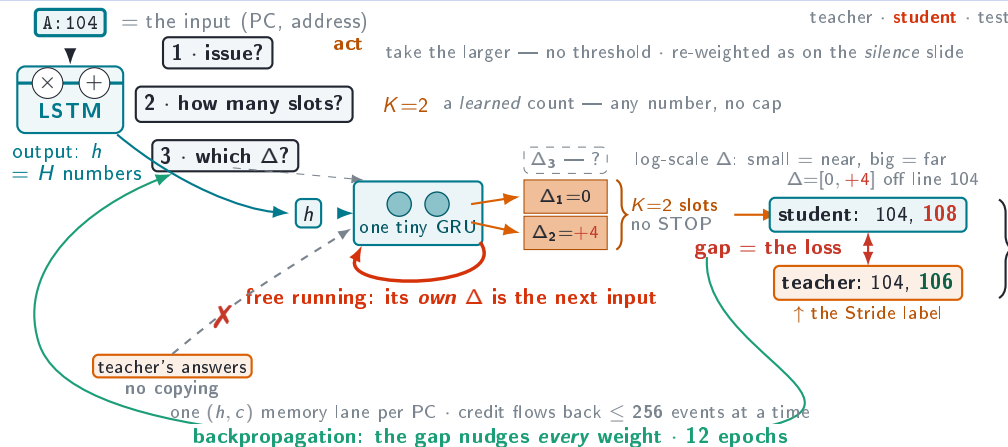
Close-Loop Training · the Student

answer, grade, nudge every weight — repeat



Close-Loop Training · the Student

answer, grade, nudge every weight — repeat



Test · Inference

frozen weights — a fresh answer per event

the **frozen** student walks the tape *once*:

teacher · student · **test**

train · 0–20M	warm · 25M	measured · 25M
A:100	B:40	A:102
	B:44	A:104
		...

Test · Inference

frozen weights — a fresh answer per event

the **frozen** student walks the tape *once*:



events arrive one at a time, in order — at event 4 it has seen only 0–4: the past alone

Test · Inference

frozen weights — a fresh answer per event

the **frozen** student walks the tape *once*:



events arrive one at a time, in order — at event 4 it has seen only 0–4: the past alone

replay.csv — one row per prefetched line:

pc,	line,	occ,	prefetch
0x401c,	104,	0,	104
0x401c,	104,	0,	106
0x77f0,	46,	1,	48
key = (PC, line, occurrence)			

occurrence counts repeats — replay finds the exact event

teacher · student · **test**

train · 0–20M warm · 25M measured · 25M

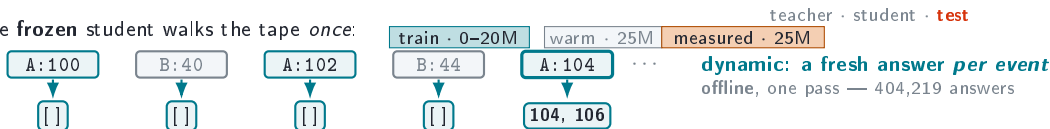
dynamic: a fresh answer per event

offline, one pass — 404,219 answers

Test · Inference

frozen weights — a fresh answer per event

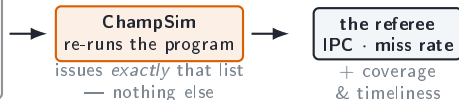
the **frozen** student walks the tape *once*:



events arrive *one at a time*, in order — at event 4 it has seen only 0-4: the *past alone*

replay.csv — one row per prefetched line:

pc,	line,	occ,	prefetch
0x401c,	104,	0,	104
0x401c,	104,	0,	106
0x77f0,	46,	1,	48
key = (PC, line, occurrence)			

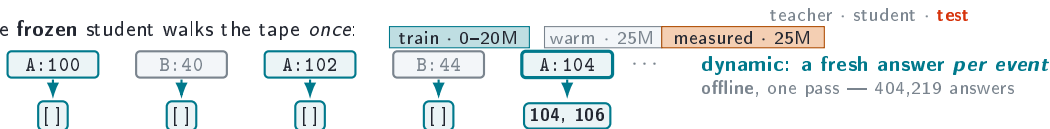


occurrence counts repeats — replay finds the exact event

Test · Inference

frozen weights — a fresh answer per event

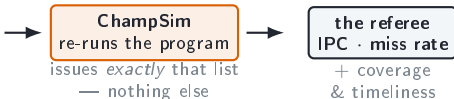
the **frozen** student walks the tape *once*:



events arrive *one at a time*, in order — at event 4 it has seen only 0–4: the *past alone*

replay.csv — one row per prefetched line:

pc,	line,	occ,	prefetch
0x401c,	104,	0,	104
0x401c,	104,	0,	106
0x77f0,	46,	1,	48
key = (PC, line, occurrence)			



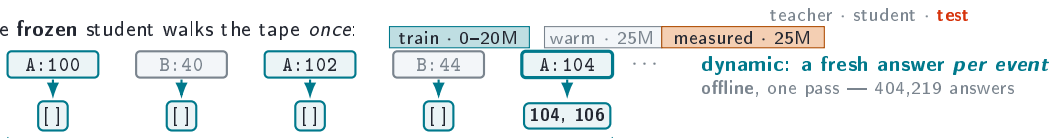
occurrence counts repeats — replay finds the *exact* event

in time? h16: **99.91% arrived in time (118,740 lines)** **0.09% late (105)**

Test · Inference

frozen weights — a fresh answer per event

the **frozen** student walks the tape *once*:



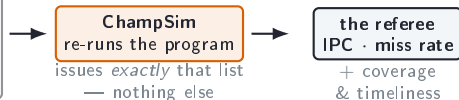
offline, one pass — 404,219 answers

events arrive *one at a time, in order* — at event 4 it has seen only 0-4: the *past alone*

replay.csv — one row per prefetched line:

pc,	line,	occ,	prefetch
0x401c,	104,	0,	104
0x401c,	104,	0,	106
0x77f0,	46,	1,	48

key = (PC, line, occurrence)



occurrence counts repeats — replay finds the *exact* event

in time? h16: **99.91% arrived in time (118,740 lines)** **0.09% late (105)**

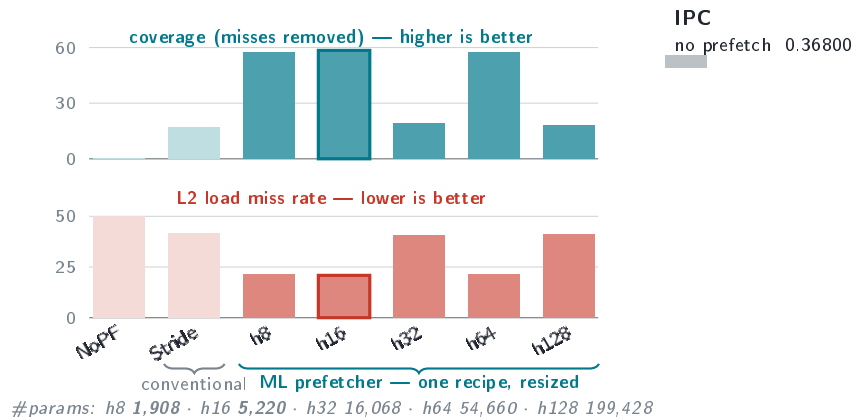
STAGE 1 · OFFLINE (this report): decide *well*?

STAGE 2 · ONLINE (next): how *fast & small*?



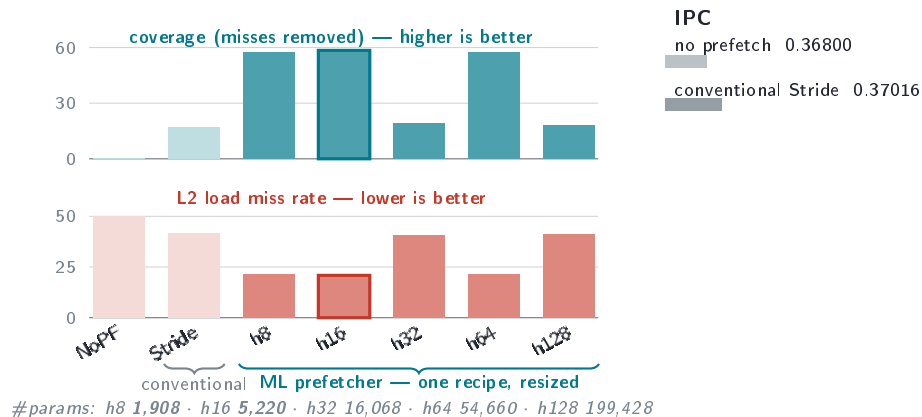
Results · Stride

the mute rule finds its voice

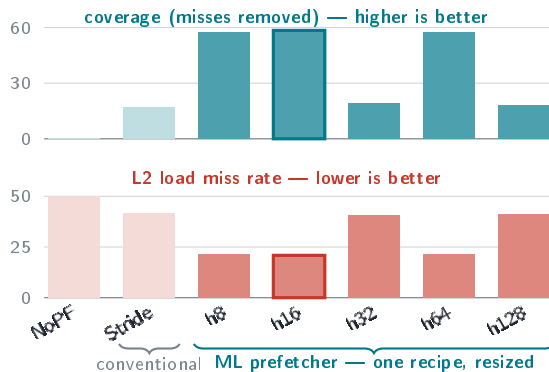


Results · Stride

the mute rule finds its voice



Results · Stride

the mute rule finds its voice

#params: h8 1,908 · h16 5,220 · h32 16,068 · h64 54,660 · h128 199,428

IPC

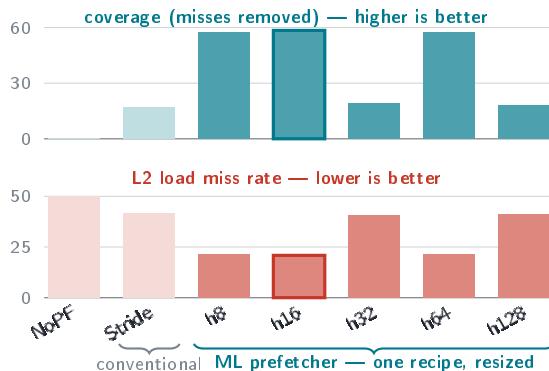
no prefetch 0.36800

conventional Stride 0.37016

LSTM h16 0.40994 +10.7% (5,220 params)

bars start at IPC 0.365

Results · Stride

the mute rule finds its voice

#params: h8 1,908 · h16 5,220 · h32 16,068 · h64 54,660 · h128 199,428

IPC

no prefetch 0.36800

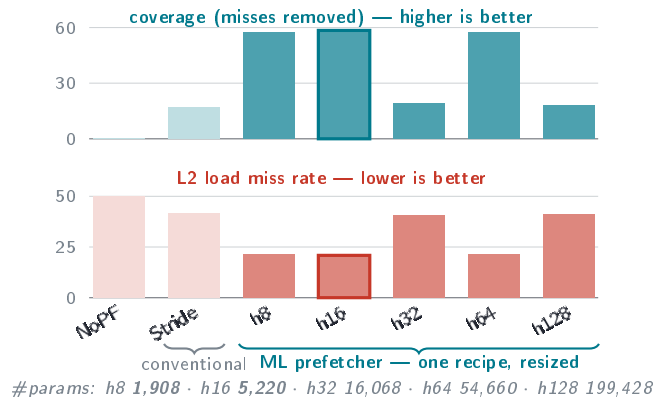
conventional Stride 0.37016

LSTM h16 0.40994 +10.7% (5,220 params)

bars start at IPC 0.365

The bill. Requests per L2 load rise from 0.30 to 0.97 — about three times the traffic.

Results · Stride

the mute rule finds its voice

IPC

no prefetch 0.36800

conventional Stride 0.37016

LSTM h16 0.40994 +10.7% (5,220 params)

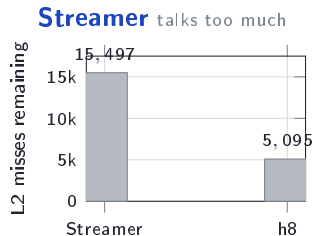
bars start at IPC 0.365

The bill. Requests per L2 load rise from 0.30 to 0.97 — about three times the traffic.

The twist. Accuracy hardly moves (28.4% → 30.2%) while coverage more than triples. The imitation score did not predict the win.

Results · Streamer, AMPM, SPP

same recipe, three different diseases



A third of the misses — half the traffic.

req/load 4.02 $\rightarrow$ 1.79 · cov.

92.4 $\rightarrow$ 97.5%

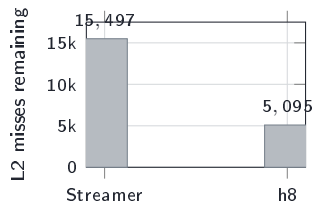
selected accuracy 11.6% $\rightarrow$ 27.4%

it already wins at h8: 2,821 params

Results · Streamer, AMPM, SPP

same recipe, three different diseases

Streamer talks too much



A third of the misses — half the traffic.

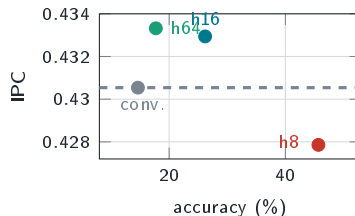
req/load 4.02 → 1.79 · cov.

92.4 → 97.5%

selected accuracy 11.6% → 27.4%

it already wins at h8: 2,821 params

AMPM the accuracy trap



The most accurate LSTM is the only one that loses.

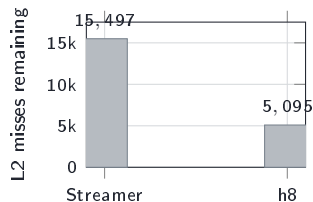
dashed line = conventional AMPM's IPC
h8 removes 88% of the misses, h16 removes 94%

params: 2,821 / 6,653 / 51,149

Results · Streamer, AMPM, SPP

same recipe, three different diseases

Streamer talks too much



A third of the misses — half the traffic.

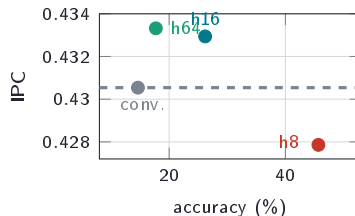
req/load 4.02 → 1.79 · cov.

92.4 → 97.5%

selected accuracy 11.6% → 27.4%

it already wins at h8: 2,821 params

AMPM the accuracy trap



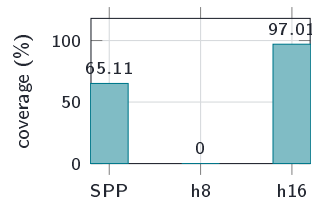
The most accurate LSTM is the only one that loses.

dashed line = conventional AMPM's IPC

h8 removes 88% of the misses, h16 removes 94%

params: 2,821 / 6,653 / 51,149

SPP aims at the wrong level



h8 sent every line to the LLC, so nothing reached L2.

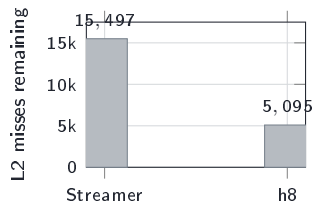
h16 · 6,609 params: miss rate 17.5% → 1.5%

IPC 0.42834 → 0.43654

Results · Streamer, AMPM, SPP

same recipe, three different diseases

Streamer talks too much



A third of the misses — half the traffic.

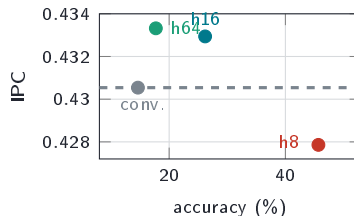
req/load 4.02 → 1.79 · cov.

92.4 → 97.5%

selected accuracy 11.6% → 27.4%

it already wins at h8: 2,821 params

AMPM the accuracy trap

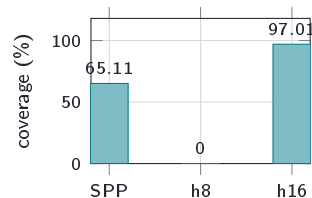


The most accurate LSTM is the only one that loses.

dashed line = conventional AMPM's IPC
h8 removes 88% of the misses, h16 removes 94%

params: 2,821 / 6,653 / 51,149

SPP aims at the wrong level



h8 sent every line to the LLC, so nothing reached L2.

h16 · 6,609 params: miss rate 17.5%
→ 1.5%

IPC 0.42834 → 0.43654

► *Stride too quiet, Streamer too loud, AMPM too cautious, SPP aimed at the wrong level — one recipe, only resized, moved all four past the hand-written originals.*

What This Shows

read the whole chain — and its limits

did it
match?

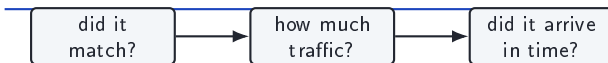
What This Shows

read the whole chain — and its limits



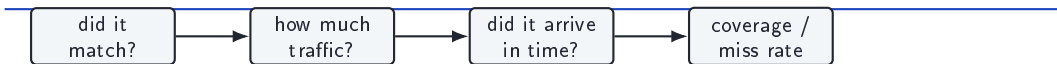
What This Shows

read the whole chain — and its limits



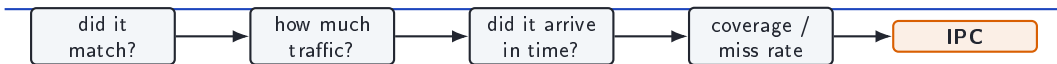
What This Shows

read the whole chain — and its limits



What This Shows

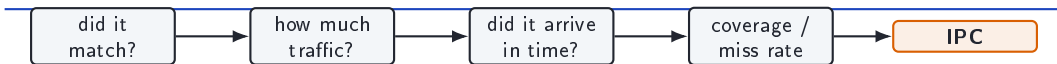
read the whole chain — and its limits



imitation stops at the first box · replay reads all five

What This Shows

read the whole chain — and its limits

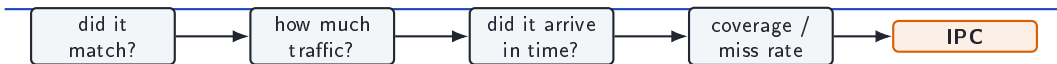


imitation stops at the first box · replay reads all five

What made it work. Raw bits and a learned memory instead of a window chosen by hand; a separate “stay silent” decision, because the labels are sparse; a learned count instead of a fixed degree; and free running, so training uses the same wire as deployment.

What This Shows

read the whole chain — and its limits



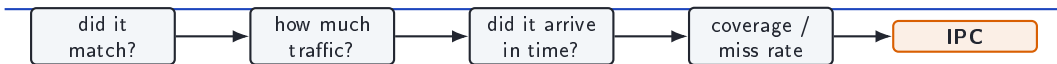
imitation stops at the first box · replay reads all five

What made it work. Raw bits and a learned memory instead of a window chosen by hand; a separate “stay silent” decision, because the labels are sparse; a learned count instead of a fixed degree; and free running, so training uses the same wire as deployment.

Scope, stated plainly. One trace, one seed, one cache level, offline replay of a recorded stream. The loss never saw IPC. Useless requests are reported as *pressure*: we did not instrument evictions, so nothing here is a claim about cache pollution.

What This Shows

read the whole chain — and its limits



imitation stops at the first box · replay reads all five

What made it work. Raw bits and a learned memory instead of a window chosen by hand; a separate “stay silent” decision, because the labels are sparse; a learned count instead of a fixed degree; and free running, so training uses the same wire as deployment.

Scope, stated plainly. One trace, one seed, one cache level, offline replay of a recorded stream. The loss never saw IPC. Useless requests are reported as *pressure*: we did not instrument evictions, so nothing here is a claim about cache pollution.

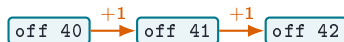
Two remembered numbers became one learned memory — same contract, same referee.

Thank you.

Appendix · Streamer

same skeleton, four changes

THE CONVENTIONAL RULE



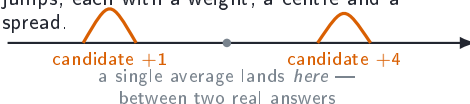
tracked **per page**, not per PC: the last offset and the last direction

when the direction repeats, it sends the next **five** lines of that page

It fires on almost every access — four prefetches per L2 load — and only about one in nine ever gets used.

WHAT CHANGES IN THE LSTM

- 1 **input** — the address only; no PC bits.
- 2 **memory** — no per-PC routing: **one** global lane, because the pattern belongs to the page, not to any single instruction.
- 3 **how many** — these labels are dense, so the hurdle collapses into a single **Poisson** count head: one learned rate.
- 4 **where** — the next line may sit at *several* plausible gaps, so each slot predicts a **four-component Gaussian mixture**: 4 candidate jumps, each with a weight, a centre and a spread.



Appendix · AMPM

a spatial map, rebuilt in time

THE CONVENTIONAL RULE

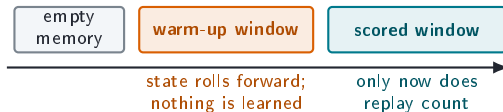


It keeps a **bitmap of the page**: which offsets have been touched, in no particular order. The map is ready the instant the page is touched.

The LSTM never sees a square — only the *order* in which they lit up. It has to rebuild that map out of time.

WHAT CHANGES IN THE LSTM

The architecture is *exactly* the Streamer LSTM. The real difference is how it is **evaluated**:



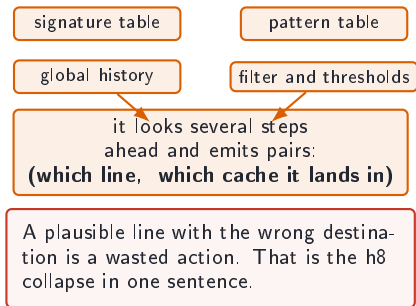
Give the LSTM no run-up and you are grading an empty memory, not the model. The warm-up only advances state — it never touches a weight.

Echo: h16 and h64 both beat conventional AMPM, while the sweep's most accurate model, h8, does not. The accuracy trap, in person.

Appendix · SPP

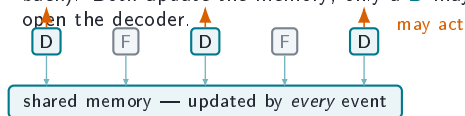
an action has two fields

THE CONVENTIONAL RULE



WHAT CHANGES IN THE LSTM

- 1 **input** — the address plus one bit saying which kind of callback this is
- 2 **two clocks in one stream** — **D** = a demand access; **F** = a line being filled (eviction feedback). *Both* update the memory; only a **D** may open the decoder.



- 3 **one more head — placement:** L2 or LLC; and the next slot is fed *both* fields

Only ~1 label in 10 says "L2". h8 dropped that minority entirely — good target scores, zero measurable benefit.